HoneyTrap: Accessible Spam Defence for React Forms

Lillie Chapman - https://github.com/lillieharlow/HoneyTrap

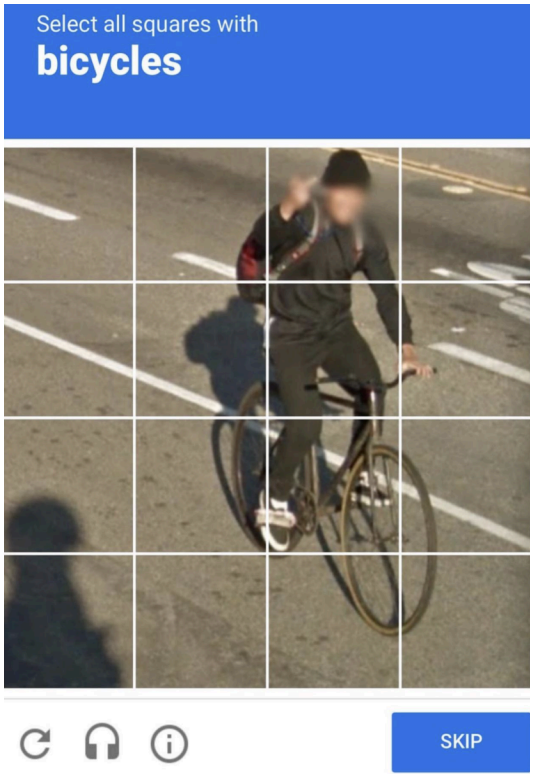

Figure 1: The captcha man on the bike flipped me off (r/funny 2025)

Stuck in CAPTCHA hell? Looking at Figure 1, trying to decide if the squares with the bicycle rider count. We've all been there; in fact, more than 67% of users abandon forms when they encounter friction, such as CAPTCHAs (Insiteful n.d.).

Form fields are an open invitation for spam to fry your data integrity. You can expect 42% of sign-up forms and 26% of contact form submissions to be spam (OOPSpam 2025). The U.S. alone generates 8 billion spam emails a day (Ramirez 2025).

The ethical fry up: spam protection or human access

Client-side, single-layered checks such as honeypot fields or timing restrictions are a good starting point for protecting your forms. However, online guides are often framework-specific, and single-layer defence approaches fall short in securely protecting your React forms against

modern bots. “With AI-driven cybercrime projected to grow by 37% annually between 2023 and 2030, relying on a single defence mechanism simply isn’t enough” (The Reform Team n.d., para. 51).

Developers understand that security against spam attacks is a multi-layered approach, implemented across the stack. Resources are often focused on server-side and database defence, with a combination of rate limiting and bot-detection services. Spam protection at these levels is highly effective, but we shouldn’t put all our spam in one pan.

Current frontend form protection relies on third-party CAPTCHA services such as reCAPTCHA or hCaptcha. These services are reliable, but they’re difficult for beginner developers to maintain and hinder human access. Creating barriers that turn basic form submission into a frustrating experience.

CAPTCHA is inherently riddled with accessibility blind spots; blind users can’t pass distorted images in screen readers, deaf users fail audio transcriptions, and motor-impaired users fail time restraints. When systems are not accessible, they exclude 1.3 billion people worldwide who live with a significant disability from basic web access (World Health Organisation 2025; World Wide Web Consortium 2025).

Accessibility-first design shouldn’t be an afterthought. Especially when you consider WebAIM’s (2025) Million report: out of the one million home pages tested, 94.8% failed to meet Web Content Accessibility Guidelines (WCAG), and 48.2% did not have form input labels. We have an ethical obligation to create accessible forms by default.

Governing bodies such as the Association for Computing Machinery (ACM) Code of Ethics and the Disability Discrimination Act 1992 (Cth) require web developers to design inclusive systems and avoid discrimination based on disability by ensuring accessibility. Further emphasising accessibility as a priority to prevent harm to all users, such as people with disabilities who face CAPTCHA barriers. If form submissions require disabled users to solve puzzles they fundamentally can’t, this problem is shifted onto society's most vulnerable users. Creating accessibility-first forms is more than just a trend; it’s a basic human right (Association for

Computing Machinery 2018, principles 1.1, 1.2, 1.4; Disability Discrimination Act 1992 (Cth), s 24).

The opportunity: serving up compliance demands without burning the user experience

UsableNet's Chief Innovation Strategist predicts "accessibility will increasingly be viewed as an indicator of overall system quality" (Taylor 2025, para. 19). Moving beyond ticking off a compliance checklist and focusing more on the user and final rendered experience. As AI continues to grow, developers are paramount in maintaining inclusive, accessible systems. Understanding that assistive technologies interact with the final rendered experience, not the source code. The future of web accessibility is determined by: "how websites are built, how content is generated and assembled, and how browsers and AI systems render experiences" (Taylor 2025, para. 29).

WebAIM's 2026 predictions define WCAG 2.2 as the baseline, a default expectation. In line with UsableNet's report, AI can not replace developers when it comes to creating accessible web content. Anticipating the same shift in respect to user preferences, the user experience will matter more than the system settings. "Concepts like clear, concise control names, predictable focus management, gesture alternatives, and logical reading order" (Northup 2025, para. 16) will push developers to a new level of detail. There will be a resurgence in the use of native HTML elements for their built-in accessibility support. Reducing complex ARIA patterns and working more predictably with assistive technology (Northup 2025).

This is the gap that HoneyTrap fills, aligning with these predictions of web accessibility, building accessibility and security together at the rendering layer. Moving away from inaccessible systems like CAPTCHA, in favour of native form controls. Using WCAG 2.2 as a baseline and focusing on a multi-layered defence that, above all, maintains an inclusive rendered user experience. No extra dependencies, no complex ARIA patterns, just a React source code kit that seamlessly integrates into any React application.

The HoneyTrap recipe: no puzzles, no CAPTCHA, no exclusion

Instead of asking users to prove they're human, HoneyTrap reveals automated spam through three layers of invisible bot detection. A security defence tool that any developer can

implement into new or existing React applications. HoneyTrap builds reusable React components and a modular custom hook, `useHumanCheck`, that bundles `useRef`, `useState`, `useEffect`, and `useCallback`. Where your form will silently observe three patterns of human behaviour that bots struggle to simulate. Now we are cooking a lot more spam with three pans working simultaneously. Frying low-effort, automated spam bots before they reach the backend.

Layer 1: The Honeypot Field

"A honeypot is a hidden field added to your web form that legitimate users never see or interact with" (Sandu 2025, para. 5). HoneyTrap coats low-level, automated spam in a layer of sticky honey that complies with WCAG 2.2 standards:

- **1.3.2 Meaningful Sequence (Level A) and 4.1.2 Name, Role, Value (Level A):** `aria-hidden="true"` removes the field from the screen reader accessibility tree, ensuring assistive technology users never encounter it.
- **2.1.1 Keyboard (Level A)**, **2.4.3 Focus Order (Level A)** and **2.4.7 Focus Visible (Level AA):** `tabIndex={-1}` removes the field from keyboard navigation, preventing keyboard-only users from accidentally interacting with it (World Wide Web Consortium 2024).

HoneyTrap combines this with CSS styling (`position: absolute; left: -10000px; opacity: 0; height: 0; width: 0`) to hide the honeypot from sighted users whilst remaining in the DOM, supporting a non-interference approach without introducing visual confusion. On form submission, a validation hook checks if the honeypot field has been filled. If filled, the submission is rejected with an error message displayed to the user.

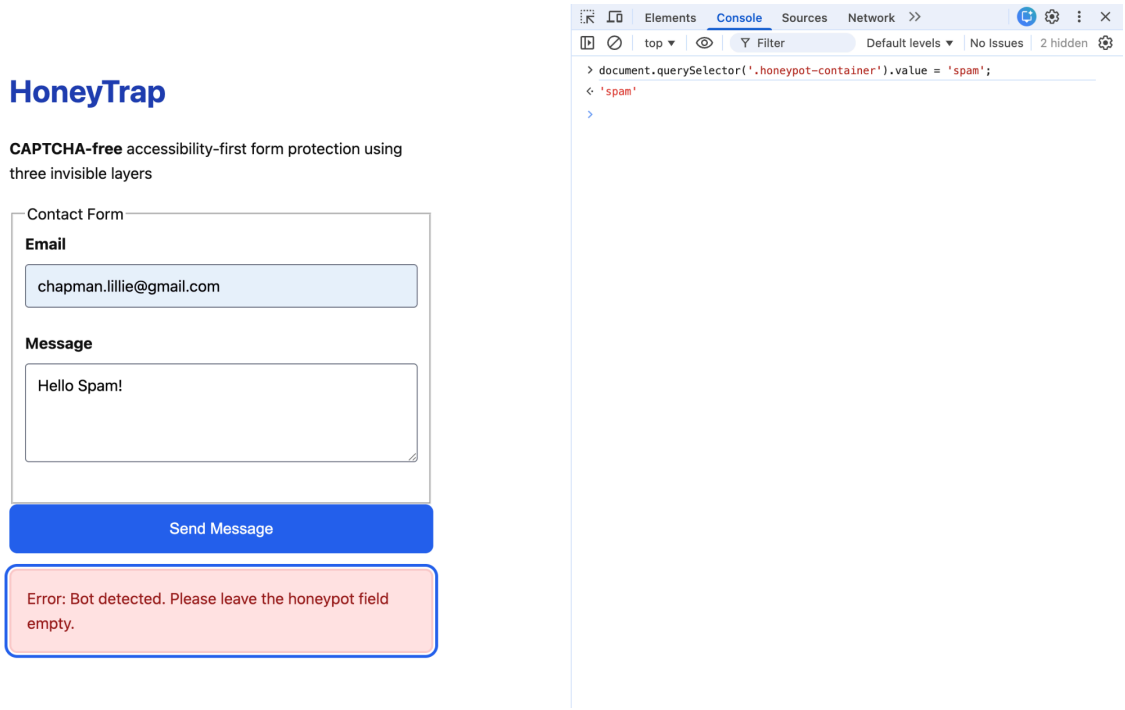

Figure 2: Screenshot of HoneyTrap honeypot field displaying error message when filled (Chapman 2026)

Layer 2: Timing Validation

Like honeypots, timing validation is not new; it's been used in form validation since the early days of jQuery in 2006 (jQuery Validation Plugin n.d.). Time-based checks are having a renaissance as a lightweight, first line of defence (Dennis n.d.). They're a seamless way to detect rapid-fire bot submissions occurring multiple times per second. When the form mounts, HoneyTrap captures a timestamp (`Date.now()`). On form submission, HoneyTrap captures that time (`const now = Date.now()`) and compares it to the first timestamp. Calculating the elapsed time (`elapsed < minTime`) and frying any submissions that come in too hot, too fast, before the minimum threshold (`minTime: 2500`), while maintaining WCAG 2.2 standards:

- **2.2.1 Timing Adjustable (Level A):** `minTime: 2500` can be tailored to specific application requirements. It is a minimum delay, not a time limit, so users are not restricted by time to complete forms.
- **3.3.1 Error Identification (Level A)** and **4.1.3 Status Messages (Level AA):** All HoneyTrap error messages are displayed and announced to screen readers with `role="alert"`

and `aria-live="assertive"`. Clarifying any issue a user may encounter (World Wide Web Consortium 2024).

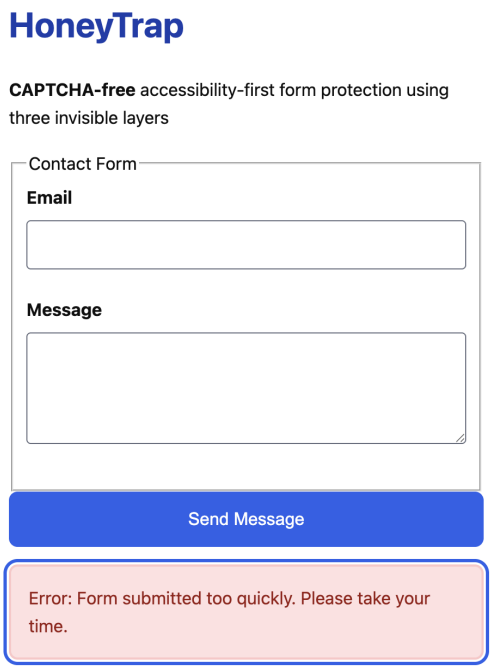

Figure 3: Screenshot of HoneyTrap timing validation displaying error message (Chapman 2026)

Layer 3: Accessibility-first Interaction Detection

The final pan in HoneyTrap's three-layer defence detects genuine human interaction. While bots can fill forms and submit them instantly, they rarely interact with the page itself. HoneyTrap tracks two interaction patterns: pointer movement (mouse or touch) and keyboard input. When a form mounts, the hook (`useHumanCheck`) uses `useEffect` to attach invisible event listeners to the document. Achieved via a custom helper (`useDocumentEvent`), which counts `pointermove` and `keydown` events. On submission, validation requires either pointer movement (10+ events) OR keyboard input (3+ key presses). This OR logic is critical for WCAG 2.2 compliance:

- **1.3.1 Info and Relationships (Level A)** and **1.3.2 Meaningful Sequence (Level A)**: Validation happens silently in the background, preserving the natural form sequence for

all users. The event listeners count interactions without using visual indicators or form controls. Both of which can disrupt reading order.

- **2.1.1 Keyboard (Level A)**, **2.4.3 Focus Order (Level A)** and **2.4.7 Focus Visible (Level AA)**: The OR logic ensures no single input method is required and either passes validation. `pointerMoves.current < 10 && keyPresses.current <3` ensures keyboard-only users can pass using only keyboard input. Touch and mouse users can pass using only pointer movements.
- **4.1.2 Name, Role, Value (Level A)** and **4.1.3 Status Messages (Level AA)**: HoneyTrap uses `role="alert"` and `aria-live="assertive"` to clarify and clearly communicate any validation error messages to screen readers (World Wide Web Consortium 2024).

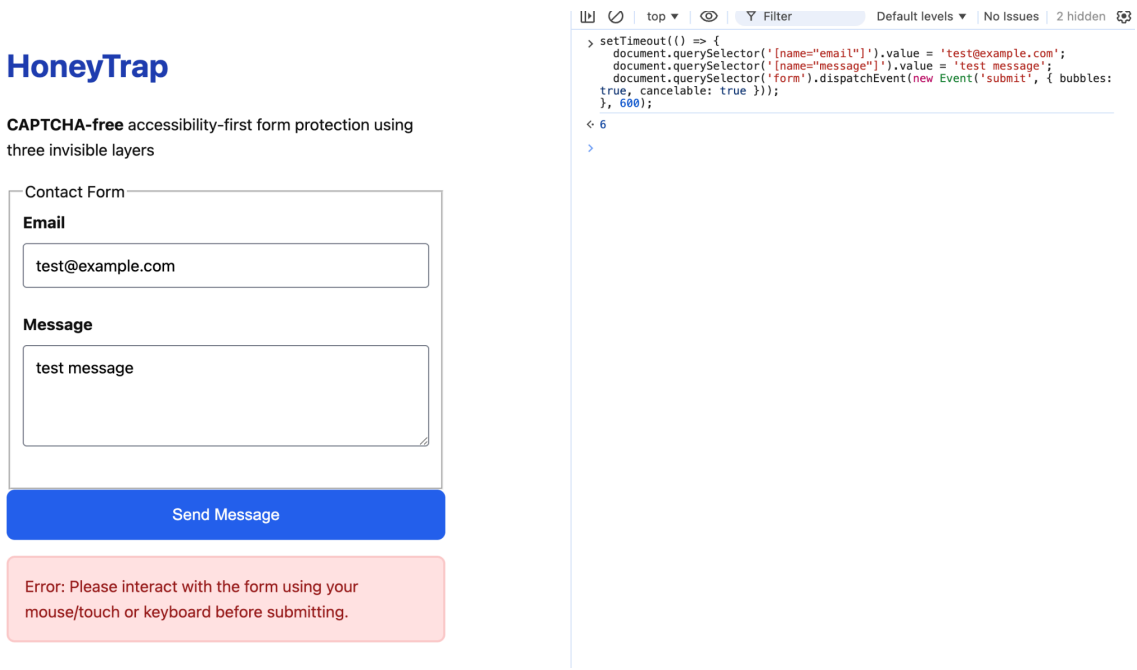

Figure 4: Screenshot of HoneyTrap interaction detection displaying error message (Chapman 2026)

Together, these three layers - honeypot field, timing validation and interaction detection create an invisible bot filter that protects your forms client-side, reduces backend load, and inclusively renders to all users, regardless of ability or device.

Other recipes: alternatives to HoneyTrap

Table 1: Alternative technology, skills and knowledge to HoneyTrap.

Alternative	HoneyTrap Uses	Key Trade-off
Vue.js / Svelte	React 19.2.0	Both frameworks can implement HoneyTrap's logic. However, React is the second-most-used framework globally (Stack Overflow 2025). Its ecosystem size and job-market relevance make it desirable for beginner developers seeking reusable form protection.
TypeScript	JavaScript + JSDoc	TypeScript adds type safety, but for beginner developers, it can be complicated and increase cognitive load. HoneyTrap keeps the entry barrier low with comprehensive JSDoc comments that modern IDEs, e.g. VS Code, can read and provide the same autocomplete hints without TypeScript complexity.
Vanilla JavaScript	React components	Vanilla JavaScript can implement HoneyTrap logic, but React components enable reusability and follow DRY principles. Their declarative nature reduces beginner confusion, as they're easier to understand than imperative DOM scripts.
reCAPTCHA / hCaptcha	Three custom layers of bot detection	CAPTCHA catches more sophisticated bots, but it systematically excludes disabled users and causes form abandonment. HoneyTrap adopts an accessibility-first design to include all users, reduce form friction, and maintain WCAG 2.2 Standards.
Class	Functional +	React hooks (`useState`, `useEffect`) are the modern

Components	Hooks	standard and offer cleaner, more composable code. Beginner React developers learn functional components with hooks as opposed to class components, which are considered legacy knowledge.
AND Logic	OR Logic	Regarding interaction type, OR logic might miss bots that simulate one interaction type, but AND logic creates accessibility failures. Keyboard-only users have zero pointer events; touch devices might not trigger keyboard events. HoneyTrap effectively uses three layers of validation whilst preserving accessibility.

Source: compiled by author (2026).

The ingredients: skills, knowledge and technology of HoneyTrap

HoneyTrap includes ready-to-use CSS styling that meets WCAG 2.2 accessibility standards (colour contrast, focus indicators, error identification, honeypot hiding and readability). You don't need to understand CSS to use HoneyTrap, but you can customise the colour values in `:root` to meet your style needs. To maintain accessibility-first design, ensure that contrast ratios are met: normal text contrast ratio ≥ 4.5:1 and UI component contrast ratio ≥ 3:1.

Table 2: Essential technologies used to integrate HoneyTrap into your existing React application.

Technology	**Required Version**	**Why?**
React	19.0+	HoneyTrap uses React Hooks (`useState`, `useEffect`, `useRef`, `useCallback`), which have been standard since React 16.8. HoneyTrap uses React 19.2.0 for modern development with the latest features.
JavaScript	ES2020+	HoneyTrap uses modern JavaScript features:

		- Optional chaining: `honeypot?.value.trim()` - Arrow functions and destructuring - **`const`**/**`let`** variables

Source: compiled by author (2026).

Table 3: Essential skills and knowledge used to integrate HoneyTrap into your existing React application.

Skill/Knowledge	**What You Will Work With**	**Why?**
React Component Nesting	Replacing `<form>` tags. Wrapping your form inputs inside `<HumanCheckForm>`.	`<HumanCheckForm>` must wrap your existing inputs to inject the honeypot field and catch the submit event. Understanding component nesting allows you to correctly place the `<HumanCheckForm>` wrapper and pass inputs as `children`.
Props & Children	Passing `onValidSubmit`, `minTime`, `honeypotName` props.	Understanding `props` allows you to customise validation behaviour. HoneyTrap's validation thresholds and callbacks are controlled via `props`. You have to pass `onValidSubmit` to receive validated form data. Optionally, you can change `minTime` and `honeypotName` values to your project's needs. Understanding `props` allows you to customise validation behaviour.

Form Handling	Understanding `onSubmit`, `e.preventDefault()` and `FormData`.	Needed to capture user input after validation. Honeytrap calls `e.preventDefault()` to stop the browser's native form submission so it can run validation and show errors. When validation passes, the `onValidSubmit` callback receives the event object. You need to extract form data using the `FormData` or the `.target.elements` property to send it to your backend.
Accessibility Basics	ARIA attributes (e.g. `aria-live`, `role="alert"`). Semantic HTML (e.g. `<label>`, `htmlFor`).	WCAG 2.2 compliance, using `aria-live="assertive"` to announce errors and `aria-hidden="true"` on the honeypot field. Any changes to ARIA attributes can break screen reader compatibility and keyboard navigation. Pairing semantic HTML `<label>` elements with `htmlFor` so form fields are announced correctly by screen readers.
Client-Side Limitations	Client-side checks will not stop all bots.	HoneyTrap blocks low-effort, automated bots at the client level, reducing backend load. Sophisticated bots can bypass HoneyTrap, so your server must re-validate: - Validate all input data - Check the honeypot field is empty - Implement rate limiting.

Source: compiled by author (2026).

The method: instructions to integrate HoneyTrap

Total time to manually integrate is approximately 48 minutes, depending on copy-paste speed, debugging, and customisation. Achieved in seven steps:

1. **Verify software requirements/dependencies (3min)**
2. **Copy-paste hook (3 min)**
3. **Copy-paste three components (9 min)**
4. **Add accessibility-first CSS (3 min)**
5. **Integrate into existing project form (15 min)**
6. **Customisation (5 min)**
7. **Verify HoneyTrap protection (10 min)**

(Demo install steps can be found here: https://github.com/lillieharlow/HoneyTrap).

1. **Verify software requirements/dependencies (3 min)**

   Inside your React project, verify that `package.json` dependencies have `"react": "^19.2.0"`

   and `"react-dom": "^19.2.0"`. The software requirement is React version 19.x or higher. If you have an older version, update it:

```bash
npm install react@^19.2.0 react-dom@^19.2.0
```

2. **Copy-paste hook (3 min)**

   Enter your `src/hooks` folder, create a new file, `useHumanCheck.js`. Copy-paste this code:

```
import { useRef, useState, useEffect, useCallback } from "react";
```

```
/**
* Helper hook to attach document event listeners with automatic
cleanup.
* @param {string} eventName - The event to listen for
* @param {Function} handler - The event handler function
*/
const useDocumentEvent = (eventName, handler) => {
 useEffect(() => {
   document.addEventListener(eventName, handler);
   return () => document.removeEventListener(eventName, handler);
 }, [eventName, handler]);
};

/**
* Custom hook that provides client-side bot detection for form
validation.
*
* Server-side validation required for actual security (client-side
can be bypassed by high-level bots).
*
* Detection layers:
* 1. Interaction - Requires pointer movement (10+) OR keyboard input
(3+ keys)
* 2. Timing - Blocks submissions faster than minimum time threshold
* 3. Honeypot - Validates hidden field remains empty
*
* @param {Object} [options] - Configuration options
* @param {number} [options.minTime=2500] - Minimum milliseconds
required before form can be submitted
```

```
* @returns {Object} Validation interface
* @returns {string|null} returns.error - Current validation error
message or null
* @returns {Function} returns.validate - Validates form:
validate(formElement, honeypotFieldName)
* @returns {Function} returns.reset - Resets all tracking counters
and errors
*
* @example
* const { error, validate, reset } = useHumanCheck({ minTime: 3000
});
*/
export function useHumanCheck({ minTime = 2500 } = {}) {
 const startTime = useRef(null);
 const pointerMoves = useRef(0);
 const keyPresses = useRef(0);
 const [error, setError] = useState(null);

 useEffect(() => {
   startTime.current = Date.now();
 }, []);

 useDocumentEvent("pointermove", () => pointerMoves.current++);
 useDocumentEvent("keydown", () => keyPresses.current++);

 const setValidationError = useCallback((message) => {
   setError(message);
   return { valid: false, error: message };
 }, []);
```

```
const validate = useCallback(
  (formElement, honeyTrap) => {
    const now = Date.now();
    if (now - startTime.current < minTime) {
      return setValidationError(
        "Form submitted too quickly. Please take your time.",
      );
    }

    const honeypot = formElement.elements[honeyTrap];
    if (honeypot?.value.trim()) {
      return setValidationError(
        "Bot detected. Please leave the honeypot field empty.",
      );
    }

    if (pointerMoves.current < 10 && keyPresses.current < 3) {
      return setValidationError(
        "Please interact with the form using your mouse/touch or " +
        "keyboard before submitting.",
      );
    }

    setError(null);
    return { valid: true, error: null };
  },
  [minTime, setValidationError],
);

const reset = useCallback(() => {
```

```
    startTime.current = Date.now();
    pointerMoves.current = 0;
    keyPresses.current = 0;
    setError(null);
  }, []);

  return { error, validate, reset };
}
```

3. **Copy-paste three components (9 min)**

   Enter your `src/components` folder, create three new files:

   1. Create `HoneypotField.jsx`. Copy-paste this code:

```
/**
* Hidden honeypot input.
*
* @component
* @param {Object} props
* @param {string} [props.name="website"] - Form field name used for
bot detection.
* @returns {JSX.Element}
*/
export default function HoneypotField({ name = "website" }) {
  return (
    <input
      type="text"
      name={name}
      tabIndex={-1}
      autoComplete="off"
      aria-hidden="true"
```

```
      className="honeypot-container"
    />
  );
}
```

2. Create `FormField.jsx`. Copy-paste this code:

```
/**
* Reusable labelled form field wrapper.
* Label is associated with input via htmlFor/id for clear semantic
connection.
*
* Accessibility: Screen readers announce labels, label click focuses
input, keyboard users interact naturally.
*
* @component
* @param {Object} props
* @param {string} props.id - Input id used for the label association.
* @param {string} props.label - Visible label text.
* @param {JSX.Element} props.children - Form control element (input,
textarea, etc).
* @returns {JSX.Element}
*/
export default function FormField({ id, label, children }) {
  return (
    <div className="form-field">
      <label htmlFor={id} className="form-label">
        {label}
      </label>
      {children}
```

```
    </div>
  );
}
```

3. Create `HumanCheckForm.jsx`. Copy-paste this code:

```
import { useRef, useState, useEffect } from "react";

import HoneypotField from "./HoneypotField";
import { useHumanCheck } from "../hooks/useHumanCheck";

/**
* Form wrapper with bot protection using three security layers:
* 1. Honeypot field (hidden, aria-hidden)
* 2. Timing check (minimum 2.5 seconds before submission)
* 3. Interaction detection (requires pointer movement 10+ times OR
keyboard input 3+ key presses)
*
* Accessible for all users: keyboard-only, screen readers, motor
disabilities, touch devices.
* Server-side validation required for actual security (client-side
can be bypassed by high-level bots).
*
* @component
* @param {Object} props
* @param {JSX.Element} props.children - Form fields to render inside
the form.
* @param {Function} [props.onValidSubmit] - Callback fired when form
passes all validation checks.
* @param {number} [props.minTime=2500] - Minimum milliseconds before
```

```
form submission is allowed.
* @param {string} [props.honeypotName="website"] - Name attribute for
the hidden honeypot field.
* @returns {JSX.Element}
*/
export default function HumanCheckForm({
 children,
 onValidSubmit,
 minTime = 2500,
 honeypotName = "website",
}) {
 const formRef = useRef(null);
 const errorRef = useRef(null);
 const [error, setError] = useState(null);
 const { validate, reset } = useHumanCheck({ minTime });

 useEffect(() => {
   reset();
 }, [reset]);

 useEffect(() => {
   if (!error) return;
   errorRef.current?.focus();
   errorRef.current?.scrollIntoView({ behavior: "smooth", block:
"center" });
 }, [error]);

 const handleSubmit = (e) => {
   e.preventDefault();
   const form = formRef.current;
```

```
  if (!form) return;

  const result = validate(form, honeypotName);
  if (!result.valid) {
    setError(result.error);
    return;
  }

  setError(null);
  onValidSubmit?.(e);
};

return (
  <form
    ref={formRef}
    onSubmit={handleSubmit}
    noValidate
    aria-describedby={error ? "form-error" : undefined}
  >
    <HoneypotField name={honeypotName} />

    {children}

    {error && (
      <div
        id="form-error"
        role="alert"
        aria-live="assertive"
        tabIndex={-1}
        className="hc-error"
```

```
          ref={errorRef}
        >
          Error: {error}
        </div>
      )}
    </form>
  );
}
```

4. **Add accessibility-first CSS (3 min)**

   Update your `src/index.css` file with this code:

```
/* Colour Variables */
:root {
 --color-primary: #2563eb;
 --color-border: #6b7280;
 --color-error: #991b1b;
 --color-error-bg: #fee2e2;
 --color-error-border: #fecaca;
}

/* Honeypot Field - makes the field invisible to users and screen
readers */
.honeypot-container {
 position: absolute;
 left: -10000px;
 opacity: 0;
 height: 0;
 width: 0;
 overflow: hidden;
```

```
}

/* Form Input Styling - font-size: 1rem prevents iOS auto-zoom.
Provides consistent, accessible form controls */
.form-input,
.form-textarea {
 width: 100%;
 padding: 0.75rem;
 border: 1px solid var(--color-border);
 border-radius: 4px;
 font-family: inherit;
 font-size: 1rem;
}

.form-textarea {
 resize: vertical;
}

/* Focus Indicators (WCAG 2.4.7) - Visible focus for keyboard
navigation */
button:focus,
input:focus,
textarea:focus {
 outline: 3px solid var(--color-primary);
 outline-offset: 2px;
}

label:focus-within {
 outline: 3px solid var(--color-primary);
 outline-offset: 2px;
```

```
}

/* Error Messages (WCAG 1.4.3, 4.1.3) - High contrast colours for
visibility. Used with aria-live regions for screen readers */
.hc-error {
 background: var(--color-error-bg);
 color: var(--color-error);
 border: 2px solid var(--color-error-border);
 border-radius: 8px;
 padding: 1rem;
 margin-top: 1rem;
}

.hc-error:focus {
 outline: 3px solid var(--color-primary);
 outline-offset: 2px;
}

/* Form Field Container - readability spacing */
.form-field {
 margin-bottom: 1.5rem;
}

.form-label {
 display: block;
 margin-bottom: 0.5rem;
 font-weight: bold;
}
```

5. **Integrate into existing project form (15 min)**

1. Import `HumanCheckForm`, `FormField` and `index.css` into your existing form component.
2. Replace the `<form>` element with `<HumanCheckForm onValidSubmit={handleSubmit}>`.
3. Replace `<input>` and `<label>` pair with `<FormField>`.
   Add `className="form-input"` to all `<input>` elements.
   Add `className="form-textarea"` to all `<textarea>` elements.

Example form component with HoneyTrap integration:

```
import HumanCheckForm from "./components/HumanCheckForm";
import FormField from "./components/FormField";
import "./index.css";

export default function ContactForm() {
 const handleSubmit = (e) => {
   const formData = new FormData(e.target);
   // Send formData to your server
 };

 return (
   <HumanCheckForm onValidSubmit={handleSubmit}>
     <fieldset>
       <legend>Contact Information</legend>
       <FormField id="name" label="Your Name">
         <input
           id="name"
           name="name"
           type="text"
           required
           className="form-input"
```

```
        />
      </FormField>
      <FormField id="email" label="Email">
        <input
          id="email"
          name="email"
          type="email"
          required
          className="form-input"
        />
      </FormField>
      <FormField id="message" label="Message">
        <textarea
          id="message"
          name="message"
          required
          className="form-textarea"
        />
      </FormField>
    </fieldset>
    <button type="submit">Send</button>
  </HumanCheckForm>
 );
}
```

6. **Customisation (5 min)**

   Optional: tailor these customisations to your project:

   1. Customise the honeypot name prop:
      - src/components/HumanCheckForm.jsx = `honeypotName = "website"`

- src/components/HoneypotField.jsx = `HoneypotField({ name = "website" }`

2. Customise the minimum time prop:
   - src/components/HumanCheckForm.jsx = `minTime = 2500`
   - src/hooks/useHumanCheck.js = `useHumanCheck({ minTime = 2500 }`
3. Customise `:root` colour values:
   - src/index.css = `--color-primary`, `--color-border`, `--color-error`, `--color-error-bg`, `--color-error-border`

   Accessible contrast ratios: normal text contrast ratio ≥ 4.5:1 and UI component contrast ratio ≥ 3:1. Check here: https://webaim.org/resources/contrastchecker/.
4. Customise error messages:
   - src/hooks/useHumanCheck.js = `"Form submitted too quickly. Please take your time."`, `"Bot detected. Please leave the honeypot field empty."`, and `"Please interact with the form using your mouse/touch or keyboard before submitting."`
   - When customising your error messages, keep them clear and descriptive.

7. Verify HoneyTrap protection (10 min)

Testing is done through your browser's DevTools (F12 or right-click, inspect, Console tab). Run your application and find your field names in the browser Console:

```
document.querySelectorAll('form input, form textarea').forEach(field => {
 console.log(`name="${field.name}"`);
});
```

Note the `name` attributes and replace the examples below with your form field names.

Test 1: Honeypot Field Detection

Replace `[name="website"]` with your honeypot field name:

```
document.querySelector('[name="website"]').value = 'spam';
document.querySelector('form').dispatchEvent(new Event('submit', {
bubbles: true, cancelable: true }));
```

Expected: Error: Bot detected. Please leave the honeypot field empty.

Test 2: Timing Validation

Replace field names and values with your form fields:

```
document.querySelector('[name="email"]').value = 'test@example.com';
document.querySelector('[name="message"]').value = 'test';
document.querySelector('form').dispatchEvent(new Event('submit', {
bubbles: true, cancelable: true }));
```

Expected: Error: Form submitted too quickly. Please take your time.

Test 3: Interaction Detection

IMPORTANT: Do not move your mouse or press keys before the form submits.

Replace field names and values with your form fields:

```
setTimeout(() => {
 document.querySelector('[name="email"]').value = 'test@example.com';
 document.querySelector('[name="message"]').value = 'test';
 document.querySelector('form').dispatchEvent(new Event('submit', {
bubbles: true, cancelable: true }));
}, 2500);
```

Expected: Error: Please interact with the form using your mouse/touch or keyboard before submitting.

Test 4: Successful Submission

Fill out your form and submit.

Expected: Form submits successfully (no HoneyTrap error messages). Your form's success behaviour depends on how you configured your form's `onValidSubmit` handler.

Reference List:

Association for Computing Machinery 2018, *ACM Code of Ethics and Professional Conduct*, viewed 26 February 2026, https://www.acm.org/code-of-ethics

Dennis, A n.d., *What Are Time-Based Access Controls (TBAC)?*, viewed 26 February 2026, https://www.lumos.com/topic/lifecycle-management-time-based-access-controls

Disability Discrimination Act 1992 (Cth) s 24, viewed 26 February 2026, https://www5.austlii.edu.au/au/legis/cth/consol_act/dda1992264/s24.html

Insiteful n.d., *Form Abandonment Statistics*, viewed 18 February 2026, https://insiteful.co/blog/form-abandonment-statistics/

jQuery Validation Plugin n.d., *jQuery Validation Plugin*, viewed 24 February 2026, https://jqueryvalidation.org/

Northup, J 2025, *2026 Predictions: The Next Big Shifts in Web Accessibility*, viewed 28 February 2026, https://webaim.org/blog/2026-predictions/

Open Replay 2025, *Honeypot Fields 101: Stop Bots Without CAPTCHAs*, viewed 18 February 2026, https://blog.openreplay.com/honeypot-fields-stop-bots/

OOPSpam 2025, *2025 Annual Spam Report*, viewed 28 February 2026, https://www.oopspam.com/2025-spam-report

Ramirez, S 2025, *Email Spam Statistics 2026: Shocking Insights and Real Risks*, viewed 24 February 2026, https://sqmagazine.co.uk/spam-statistics/

r/funny 2025, *The captcha man on the bike flipped me off*, online image, viewed 17 February 2026, https://www.reddit.com/r/funny/comments/1lca68z/the_captcha_man_on_the_bike_flipped_me_off/#lightbox

Sandu, B 2025, *What Is A Honeypot And How To Implement It In A Form*, viewed 20 February 2026, https://ivyforms.com/blog/what-is-a-honeypot/#What_Is_A_Honeypot

Stack Overflow 2025, *Technology*, viewed 18 February 2026, https://survey.stackoverflow.co/2025/technology

Taylor, J 2025, *Web Accessibility in 2026: Five Predictions Shaped by How the Web Is Changing*, viewed 25 February 2026, https://blog.usablenet.com/web-accessibility-2026-predictions

The Reform Team n.d., *How to Stop Bots Submitting Forms,* viewed 20 February 2026, https://www.reform.app/blog/how-to-stop-bots-submitting-forms

WebAIM 2025, *The WebAIM Million*, viewed 14 February 2026, https://webaim.org/projects/million/

World Health Organisation 2025, *WHO Disability Health Equity Initiative*, viewed 14 February 2026, https://www.who.int/initiatives/disability-health-equity-initiative

World Wide Web Consortium 2022, *Inaccessibility of CAPTCHA*, viewed 25 February 2026, https://w3c.github.io/captcha-accessibility/

World Wide Web Consortium 2024, *Web Content Accessibility Guidelines (WCAG) 2.2*, viewed 20 February 2026, https://www.w3.org/TR/WCAG22/

World Wide Web Consortium 2025, *Understanding SC 1.1.1: Non-text Content (Level A)*, viewed 18 February 2026, https://www.w3.org/WAI/WCAG21/Understanding/non-text-content.html

Yadav, R 2024, *Honeypot for Spam Prevention React & Tailwind CSS,* viewed 14 February 2026, https://dev.to/rohitnirban/honeypot-for-spam-prevention-react-tailwind-css-57lh

Zapala, T 2024, *Stop bots from submitting your forms using a honeypot field*, viewed 14 February 2026, https://www.tymzap.com/blog/stop-bots-from-submitting-your-forms-using-a-honeypot-field